Deploying Web Applications on Containers on AWS

LOG8415E – Advanced Concepts of Cloud Computing
Lab Assignment 2 – Fall 2026

November 10, 2026

Group Members

Name	Student ID
Wilfred poirier	2270349
Rosalie Lamoureux	2264804
Calvin Cantin	2269675
Timothy Lau	2293503

1 Introduction

This report describes a containerized web application deployed on AWS EC2. The system is made of one *orchestrator* and two *workers*. Users send JSON requests to the orchestrator on port 80. The orchestrator forwards each request to a free worker (a Flask application running in a Docker container on port 5000), returns the worker's answer to the user, and queues requests when both workers are busy. Each worker performs sentiment analysis with a lightweight DistilBERT model. The whole infrastructure is created with Terraform and the software is deployed with scripts, so the system can be launched and tested with minimal manual intervention.

2 AWS Setup

2.1 Instances

All resources are created in the `us-east-1` region, in the account's default VPC, with Terraform (`terraform/main.tf`). Exactly three Ubuntu instances are deployed:

Name tag	Role	Type	Root volume
`tp2-orchestrator`	Orchestrator (port 80)	`t2.large`	20 GB
`tp2-worker1`	Worker (port 5000)	`t2.large`	30 GB
`tp2-worker2`	Worker (port 5000)	`t2.large`	30 GB

The instance type is `t2.large`, the default specified in the assignment (variable `instance_type`). The AMI is the most recent Canonical Ubuntu 22.04 (Jammy) amd64 server image, selected dynamically with a Terraform `aws_ami` data source (owner `099720109477`). Workers have a larger 30 GB disk because the PyTorch-based Docker image is large. An SSH key pair (`log8415e-tp2-key`, ed25519), generated locally by `deploy.sh`, is registered in AWS from its public key so that the deployment script can connect to the instances. No credentials are stored in the project: Terraform reads them from `~/.aws/credentials`.

2.2 Security group

A single security group, `log8415e-tp2-sg`, is attached to the three instances and opens only the required ports:

Port	Protocol	Source	Purpose
22	TCP	`allowed_ssh_cidr` variable	SSH for deployment
80	TCP	`0.0.0.0/0`	Orchestrator HTTP API
5000	TCP	the security group itself	Orchestrator → workers

Port 5000 is therefore *not* reachable from the Internet: only instances of the same security group (i.e., the orchestrator) can contact the workers. Egress traffic is unrestricted so that instances can download Docker, Python packages, and the model.

2.3 Docker installation at boot

Each instance runs the same `user_data.sh` script at first boot. It installs Docker Engine and the Compose plugin with the official "Install using the Apt repository" method (adds Docker's GPG key and apt repository, then installs `docker-ce`, `docker-ce-cli`, `containerd.io`, `docker-buildx-plugin`, and `docker-compose-plugin`), enables the Docker service, adds the `ubuntu` user to the `docker` group, and finally creates the marker file `/tmp/docker-ready`. The deployment script polls for this file to know when an instance is ready to receive the application.

2.4 Terraform outputs

After `terraform apply`, Terraform prints the orchestrator's public IP, and the public and private IPs of the two workers (`output.tf`). The orchestrator uses the workers' *private* IPs (inside the VPC), which is what allows port 5000 to stay closed to the outside.

3 Architecture

Figure 1 shows the architecture.

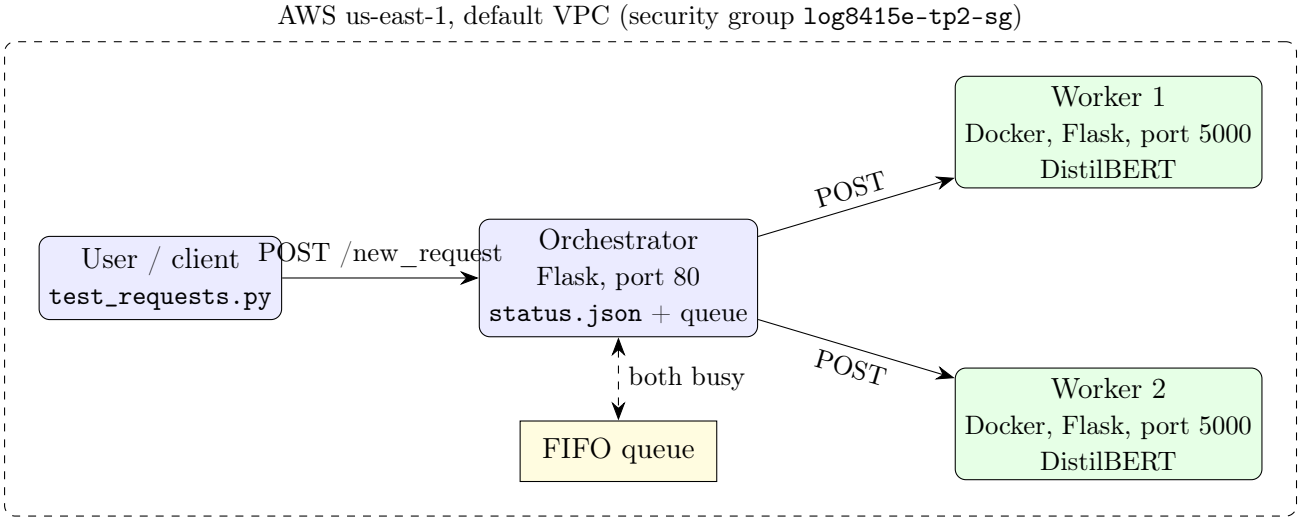


Figure 1: System architecture: one orchestrator and two single-container workers.

The request flow is:

1. A client sends `POST /new_request` with a JSON body to the orchestrator on port 80.

2. The orchestrator inspects its worker status (`status.json`) and tries to claim a free worker.
3. If a worker is `free`, it is marked `busy` and the request is forwarded to it on port 5000.
4. The worker runs the inference and returns a JSON response.
5. The orchestrator returns the response to the client. If requests are waiting, the worker immediately takes the oldest one; otherwise it is marked `free` again.
6. If both workers are `busy`, the request is placed in a FIFO queue and sent to the first worker that becomes free.

4 Docker and Docker Compose Configuration

Docker Engine and the Compose plugin are installed on every instance by `user_data.sh` (Section 2). A **single** `docker-compose.yml` is used for the whole project: it describes the two workers and the orchestrator, and each instance only starts the service(s) it is responsible for.

Listing 1: `docker-compose.yml`

```
services:
  worker1:
    build: ./worker
    image: assignment2-worker
    container_name: worker1
    environment:
      WORKER_NAME: worker1
    ports:
      - "${WORKER1_PORT:-5001}:5000"

  worker2:
    build: ./worker
    image: assignment2-worker
    container_name: worker2
    environment:
      WORKER_NAME: worker2
    ports:
      - "${WORKER2_PORT:-5002}:5000"

  orchestrator:
    build: ./orchestrator
    container_name: orchestrator
    ports:
      - "80:80"
    environment:
      WORKER1_IP: ${WORKER1_IP:-worker1}
      WORKER2_IP: ${WORKER2_IP:-worker2}
    restart: unless-stopped
```

Workers. Both workers are built from the same `./worker` directory (same image, `assignment2-worker`); only the environment variable `WORKER_NAME` differs, which is how a worker identifies itself in its JSON answers. Inside the container, the Flask application always listens on **port 5000**. On the AWS deployment, each worker instance runs **exactly one container**: `deploy.sh` copies the `worker` directory and the compose file to the instance, writes a `.env` file with `WORKER1_PORT=5000` and `WORKER2_PORT=5000`, and runs `docker compose up -d -build worker1` (respectively `worker2`). The `.env` override makes the host port equal to 5000 on the real workers. The default host ports

5001 and 5002 exist so that the two workers can run side by side on a single machine (e.g., for local development) without a port conflict.

Orchestrator. The orchestrator is also containerized (built from ./orchestrator) and publishes port 80 of the container on port 80 of its instance. The addresses of the workers are given by the environment variables WORKER1_IP and WORKER2_IP. By default they are the compose service names (worker1, worker2), which works locally thanks to Docker's internal DNS. On AWS, deploy.sh writes the workers' *private* IP addresses (read from the Terraform outputs) in the .env file before running docker compose up -d -build orchestrator. The orchestrator has the restart policy unless-stopped, so it is restarted automatically if it crashes or the instance reboots.

Dockerfiles. Both images are based on python:3.11-slim, use /app as working directory, install their Python dependencies from a requirements.txt with pip install -no-cache-dir (copied before the source code so that Docker's layer cache avoids re-installing dependencies when only the code changes), and then copy the application source.

Listing 2: Orchestrator Dockerfile

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app.py .
EXPOSE 80
# ONE process (shared in-memory queue) with many threads (concurrent requests)
CMD ["gunicorn", "-w", "1", "--threads", "64", "--timeout", "700", "-b", "0.0.0.0:80",
     "app:app"]
```

The orchestrator is served by **gunicorn** with a *single* worker process (-w 1) and 64 threads. This choice is deliberate: the worker-status table and the request queue live in the orchestrator's memory, so they must be shared by all requests, which is only the case inside one process. Concurrency is obtained with threads: each incoming request is handled by its own thread, so many requests can wait in the queue at the same time while the others are being processed. The timeout is set to 700 s so that requests waiting a long time in the queue are not killed by gunicorn.

Listing 3: Worker Dockerfile

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY worker.py .
EXPOSE 5000
CMD ["python", "worker.py"]
```

The worker image exposes port 5000 and starts worker.py directly. The Dockerfile contains no step that downloads the model, so the DistilBERT weights are not baked into the image: they are fetched and loaded when the container starts. This keeps the image smaller, but it explains why deploy.sh waits for the /health endpoint before running the tests (the first start takes longer).

Because the worker image contains PyTorch it is large, which is why the worker instances have 30 GB root volumes instead of the 20 GB used by the orchestrator. The worker image is built in parallel on both workers, since the build is the slowest part of the deployment.

5 Orchestrator: Selecting a Free Worker

The orchestrator is a Flask application (`orchestrator/app.py`) served by gunicorn on port 80. It exposes two endpoints: `POST /new_request`, which processes a request, and `GET /status`, which returns the current worker states and the queue length (useful for debugging and demonstrations).

5.1 Worker status file

The state of the workers is stored in `status.json`, using the format suggested in the assignment. The file is created when the orchestrator starts (function `init_status`): the addresses come from the environment variables `WORKER1_IP` and `WORKER2_IP`, which `deploy.sh` sets to the workers' *private* IPs, and both workers start as `free`.

```
{
  "worker1": {"ip": "<private_ip_worker1>", "port": "5000", "status": "free"},
  "worker2": {"ip": "<private_ip_worker2>", "port": "5000", "status": "busy"}
}
```

Every update writes a temporary file and then renames it with `os.replace`, which is atomic: a reader can never see a half-written file.

5.2 Claiming a worker

A single global lock (`threading.Lock`) protects both the status file and the request queue. Selecting a worker is done by `claim_worker`, always called with the lock held:

```
def claim_worker():
    """Return the name of a free worker and mark it busy, or None."""
    status = read_status()
    for name, w in status.items():
        if w["status"] == "free":
            w["status"] = "busy"
            write_status(status)
            return name
    return None
```

The workers are scanned in order (`worker1`, then `worker2`) and the first `free` one is marked `busy` and returned; when both are free, `worker1` is therefore preferred. Because the check and the update happen in the same critical section, two concurrent requests can never claim the same worker.

5.3 Forwarding and answering

If a worker was claimed, `new_request` starts a thread that runs `run_on_worker`. This thread sends an HTTP POST with the client's JSON body to `http://<worker_ip>:5000/process` (timeout: 120 s). Meanwhile the thread handling the client's HTTP connection waits (up to 600 s) until the job is done, and then answers the client with a JSON object containing: `worker` (which worker processed the job), `queued`, `wait_in_queue_s`, and `worker_response` (the worker's JSON output, unchanged). The HTTP status code of the worker is propagated.

Errors are reported explicitly: invalid or missing JSON gives `400`; a worker that is down, times out, or returns invalid JSON gives `502` with the error message; and a client that waited more than 600 s gets `504`.

6 Orchestrator: Handling Queued Requests

If `claim_worker` returns `None`, both workers are busy. The request is then marked as queued (`queued = true`) and appended to `pending`, a FIFO `collections.deque`, still inside the same critical section. The client's connection is kept open and its thread simply blocks on a `threading.Event` until its job is finished, so from the client's point of view the call just takes longer.

There is no separate dispatcher: queued requests are drained by the threads that run on the workers. After a worker has finished a job, `run_on_worker` checks the queue before releasing the worker:

```
def run_on_worker(name, job):
    """Process ‘job‘ on worker ‘name‘, then keep draining the queue."""
    while True:
        job.worker = name
        job.started_at = time.time()
        try:
            w = read_status()[name]
            r = requests.post(f"http://{w['ip']}:{w['port']}/process",
                              json=job.payload, timeout=WORKER_TIMEOUT)
            job.code, job.response = r.status_code, r.json()
        except Exception as exc:        # worker down, timeout, bad JSON...
            job.code, job.response = 502, {"error": str(exc)}
        job.done.set()                  # wake up the waiting client thread

        with lock:
            if pending:                 # a request is waiting: take it,
                job = pending.popleft() # the worker stays busy
                continue
            set_status(name, "free")    # nothing waiting: free the worker
            return
```

Hence, as soon as a worker finishes, the *oldest* queued request is sent to it immediately (FIFO), and the worker is marked `free` only when the queue is empty. A queued request is therefore always eventually processed, as long as workers respond; it is not simply stored.

No lost request. A new request checks the workers and appends to the queue under the lock, and a finishing worker checks the queue and frees itself under the same lock. Therefore, either the new request sees the worker free and claims it, or it is already in the queue when the worker looks at it: a request can never stay in the queue while a worker is idle.

Measuring the wait. `wait_in_queue_s` is the time between the arrival of the request (`enqueued_at`) and the moment a worker starts processing it (`started_at`); it is close to 0 for requests served immediately.

Verification. The behaviour was verified with `test_requests.py`, which sends six concurrent requests, each asking the worker to simulate a 3-second processing time. With two workers, two requests start immediately and four must be queued, so about three rounds of 3 s are expected, i.e., about 9 s of wall-clock time. The measured results (Section 8) match this expectation.

7 Worker Application and ML Inference

Each worker is a Flask application (`worker/worker.py`) running in a Docker container and listening on port 5000. It identifies itself with the environment variable `WORKER_NAME` (`worker1` or `worker2`, set in the compose file).

Model. The worker uses the Hugging Face `transformers` pipeline `sentiment-analysis` with the lightweight `distilbert-base-uncased-finetuned-sst-2-english` model. The model is created once, when the module is imported at container start-up, and then reused for all requests. The first start therefore takes longer (the weights are downloaded and loaded), which is why `deploy.sh` waits for the health endpoint before running tests.

Endpoints.

- `GET /health` returns `{"status": "ok", "worker": "<name>"}`; it is polled by the deployment script.
- `POST /process` receives a JSON body with a `text` field and an optional `simulate_delay` (seconds). If `text` is missing, the worker answers `400` with an error message. Otherwise it sleeps for `simulate_delay` seconds (default 0), runs the classifier on the text (input truncated to the model's maximum length), and returns the input, the predicted label (`POSITIVE` or `NEGATIVE`), and its confidence score rounded to four decimals, together with the worker's name.

The `simulate_delay` field exists only for testing. Real inference on a short sentence takes a fraction of a second, so without this delay the two workers would almost never be busy at the same time and the queueing logic could not be demonstrated.

The ML inference was fully supported on the `t2.large` instances, so the approved text-processing fallback was **not** needed.

8 Example Request and Response

A single request can be sent with `curl` (replace the IP by the orchestrator's public IP printed by Terraform):

```
curl -X POST http://<ORCHESTRATOR_PUBLIC_IP>/new_request \
     -H "Content-Type: application/json" \
     -d '{"text": "I hate waiting", "simulate_delay": 3}'
```

Response (taken from our test run in `responses.json`):

```
{
  "queued": false,
  "wait_in_queue_s": 0.002,
  "worker": "worker1",
  "worker_response": {
    "input": "I hate waiting",
    "result": {"label": "NEGATIVE", "score": 0.9945},
    "worker": "worker1"
  },
  "total_time_s": 3.27
}
```

8.1 Concurrency test: queue in action

Running `python3 test_requests.py <ORCHESTRATOR_PUBLIC_IP>` sends six concurrent requests with a simulated 3-second delay. The results were:

Input text	Worker	Queue wait (s)	Total (s)	Result
I hate waiting	worker1	0.002	3.27	NEGATIVE (0.9945)
The demo failed again	worker2	0.004	3.28	NEGATIVE (0.9998)
This is terrible.	worker2	3.165	6.34	NEGATIVE (0.9996)
Cloud computing is fun	worker1	3.137	6.34	POSITIVE (0.9998)
I love this course!	worker2	6.187	9.38	POSITIVE (0.9999)
What a wonderful day	worker1	6.154	9.38	POSITIVE (0.9999)

Two requests were served immediately (queue wait ≈ 0 s), two waited about one round (≈ 3 s) and two waited about two rounds (≈ 6 s). The total times of 3.3, 6.3 and 9.4 s correspond to three successive rounds on two workers, which confirms that (i) both workers are used in parallel, (ii) no request is lost, and (iii) queued requests are dispatched as soon as a worker becomes free. The roughly 0.3 s of overhead per round is network and inference time.

9 Instructions to Run the Code

9.1 Prerequisites

On the local machine: Terraform, Python 3, a Bash shell with `ssh`/`scp`, and the AWS Lab credentials configured in `~/.aws/credentials` (never included in the archive). The SSH key pair does not need to be created by hand: `deploy.sh` generates an ed25519 key pair in `keys/` the first time it runs, and Terraform registers its public part (`keys/lab-key.pub`) in AWS.

9.2 One-command deployment

```
./deploy.sh
```

The script performs the following steps (it stops at the first error, `set -euo pipefail`):

1. Generates the SSH key if it does not exist.
2. Runs `terraform init` and `terraform apply`, then reads the orchestrator's public IP and the workers' public and private IPs from the Terraform outputs.
3. Waits, for each instance, until SSH works and `/tmp/docker-ready` exists (i.e., `user_data.sh` has finished installing Docker).
4. Copies the `worker` directory and `docker-compose.yml` to each worker and starts its container (`docker compose up -d -build`); the two workers are deployed *in parallel* because the image build is slow.
5. Copies the `orchestrator` directory and compose file to the orchestrator instance and starts the orchestrator container, configured with the workers' private IPs.
6. Waits until both workers answer on their `/health` endpoint (checked *from the orchestrator*, since port 5000 is not reachable from the Internet); this guarantees that the model is loaded before any request is sent.
7. Saves the orchestrator's address in `orchestrator_ip.txt` and runs `test_requests.py`, which sends six concurrent requests and stores the answers in `responses.json`.

Expected runtime: a full deployment from scratch takes approximately 10 minutes, mostly spent on the instance boot, the Docker installation, the worker image build, and the model load; the test itself then takes about 9 s.

9.3 Sending a test request manually

```
python3 test_requests.py $(cat orchestrator_ip.txt)

curl -X POST http://$(cat orchestrator_ip.txt)/new_request \
     -H "Content-Type: application/json" \
     -d '{"text": "I love this course!"}'
```

9.4 Cleanup

To avoid consuming AWS credits, destroy everything when done:

```
cd terraform
terraform destroy -auto-approve
```

10 Assumptions, Limitations, and Problems Encountered

10.1 Assumptions

- The `t2.large` instance type (assignment default) is used, as no other type was specified.
- The AWS Lab environment provides a default VPC in `us-east-1` and allows Terraform to create key pairs and security groups.
- Clients send JSON with a `text` field; `simulate_delay` is optional and used only to demonstrate the queue.

10.2 Limitations

- **Single point of failure:** there is a single orchestrator; if it crashes, queued requests are lost, as the queue lives in its memory.
- **No health checks or retries:** a worker error or timeout (120 s) is returned to the client as a `502`; the request is not retried on the other worker, and a dead worker is simply marked `free` again and may receive new requests that will fail the same way.
- **Capacity of the queue:** the orchestrator handles each request in one of 64 threads, so at most 64 requests can be in the system (running or waiting) at the same time; further connections wait to be accepted. A request waiting more than 700 s (the gunicorn timeout) is aborted.
- **Worker server:** the workers are started with `python worker.py`, i.e., Flask's built-in development server, which is sufficient here because the orchestrator sends at most one request at a time to each worker, but is not meant for production use (a server such as gunicorn would be).
- **Status file:** `status.json` lives in the orchestrator container's file system and is reinitialized (all workers `free`) at each start. A single file on one machine is simple but does not scale to several orchestrators.
- **Security:** SSH is open to `0.0.0.0/0` by default (variable `allowed_ssh_cidr`); in production it should be restricted to the user's own IP (`x.x.x.x/32`). Traffic between client and orchestrator is plain HTTP, without authentication.
- **Restart policy:** only the orchestrator container has `restart: unless-stopped`; the worker containers do not restart automatically after a crash or reboot.

- **Deployment script:** SSH host-key checking is disabled (`StrictHostKeyChecking=no`) because the instances are new at each deployment, and the readiness loops have no maximum wait time.
- **Abandoned requests:** if a client gives up (or receives `504` after 600 s), its request stays in the queue and is still processed by a worker, which wastes capacity.
- **Public IP changes:** instance public IPs change at every deployment, so the IPs are read from Terraform outputs each time.

10.3 Problems encountered and design challenges

We did not run into any blocking problem during the development and the deployment of the system. Several points, however, are known pitfalls of this kind of deployment and were handled explicitly in the design:

- **Docker not ready when the script connects.** A new instance accepts SSH connections before `user_data.sh` has finished installing Docker. The script therefore waits for the marker file `/tmp/docker-ready` instead of relying on SSH availability alone.
- **Slow worker start-up.** The worker image is large and the model is downloaded and loaded at container start. Sending requests too early would fail, so `deploy.sh` builds the two workers in parallel and then polls `/health` on both before running the test.
- **Race conditions in the orchestrator.** With concurrent requests, two threads could claim the same worker, or a request could be queued just after the last worker became idle. A single lock around the status file and the queue, a single gunicorn process, and the check-then-free logic in `run_on_worker` rule both cases out (Sections 5 and 6).
- **Demonstrating the queue.** Sentiment inference takes a fraction of a second, so the workers are almost never busy at the same time. We added the optional `simulate_delay` parameter so that the queue can be exercised and measured.
- **Keeping the workers private.** Port 5000 is open only to the security group itself, so the orchestrator must reach the workers through their private IPs; `deploy.sh` reads them from the Terraform outputs and passes them to the orchestrator container.

References and Use of External Tools

- Docker documentation, "Install Docker Engine on Ubuntu – Install using the apt repository", https://docs.docker.com/engine/install/ubuntu/.
- Terraform AWS provider documentation, https://registry.terraform.io/providers/hashicorp/aws/latest/docs.
- Hugging Face Transformers documentation, https://huggingface.co/docs/transformers.
- Flask documentation, https://flask.palletsprojects.com/.